

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
```

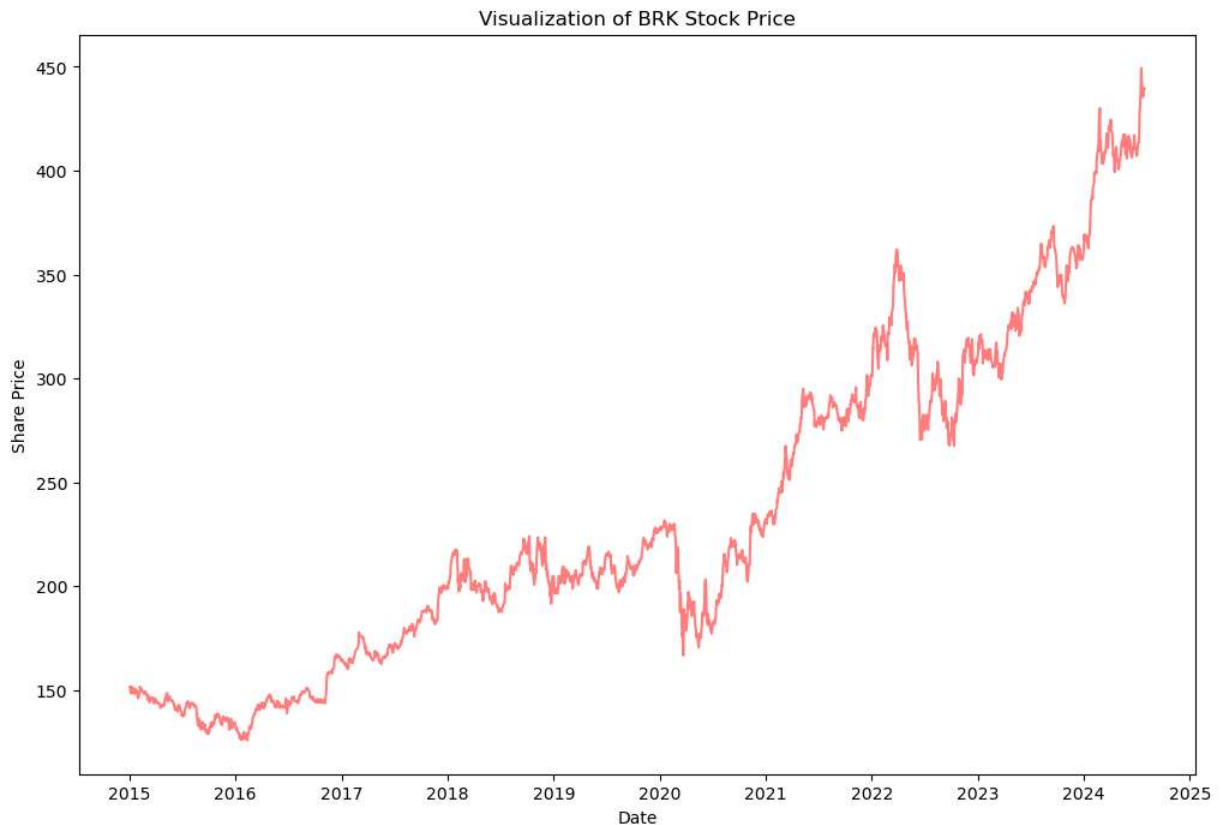
```
In [2]: #Simple plot function to visualize a stock's price over time
def basic_stock_plot(stock_df, stock_ticker):
    plt.figure(figsize=(12,8))
    plt.plot(stock_df['Date'], stock_df['High'], color = 'red', alpha=0.5)
    title_string = 'Visualization of ' + stock_ticker + ' Stock Price'
    plt.title(title_string)
    plt.xlabel('Date')
    plt.ylabel('Share Price')
    plt.show()

#Format and print a table to compare actual vs. predicted prices
def comp_table(actual_prices, predicted_high_prices, stock_ticker):
    comparison = pd.DataFrame({'Actual': actual_prices.flatten(), 'Predicted': pred
    table_title = stock_ticker + ' Prediction Comparison'
    print(table_title)
    print(comparison)
```

```
In [3]: #Import the training dataset
training_data = '../data/BRK.csv'
dataset = pd.read_csv(training_data)

#Drop rows with missing values and convert date to datetime format
clean_df = dataset.dropna()
clean_df['Date'] = pd.to_datetime(clean_df['Date'])

#Visualize the stock price
basic_stock_plot(clean_df, 'BRK')
```



```
In [4]: #Select the following columns to be features and select all rows except very last row
x = clean_df[['Open', 'Low', 'Close', 'Adj Close', 'Volume']].head(-1)
x = x.values

#Select 'High' price to be target value. Select all but first row.
y = clean_df[['High']].tail(-1)
y = y.values

#The goal here is to have all features (x vector) used to make predictions on the future
print('Shape of feature vector: ', x.shape)
print('Shape of target vector:', y.shape)
```

Shape of feature vector: (2407, 5)

Shape of target vector: (2407, 1)

```
In [5]: #Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

#Initialize linear model and fit training data to model
model = LinearRegression()
model.fit(x_train, y_train)
```

```
Out[5]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [6]: #Make predictions on testing data
predicted_high_prices = model.predict(x_test)
```

```

#Printing R2 score for training data. Very high score (close to 1) indicates model
#variability, indicating a good fit
print('Berkshire Hathaway training data R2 Score:', model.score(x_train, y_train))

#Print R2 Score comparing predicted vs actual prices. Very high score (close to 1)
print('Berkshire Hathaway testing data R2 Score:', r2_score(y_test, predicted_high_

#Print Berkshire Hathaway Comparison Table
print()
comp_table(y_test, predicted_high_prices, 'BRK')

```

Berkshire Hathaway training data R2 Score: 0.9993313549904053

Berkshire Hathaway testing data R2 Score: 0.9991274687667783

BRK Prediction Comparison

	Actual	Predicted
0	363.390015	362.522597
1	230.000000	233.191399
2	179.720001	179.848990
3	270.709991	271.838450
4	208.479996	208.090817
..	...	...
477	184.240005	187.991508
478	289.399994	288.888016
479	294.000000	290.414419
480	142.000000	142.889541
481	220.000000	220.877096

[482 rows x 2 columns]

In []:

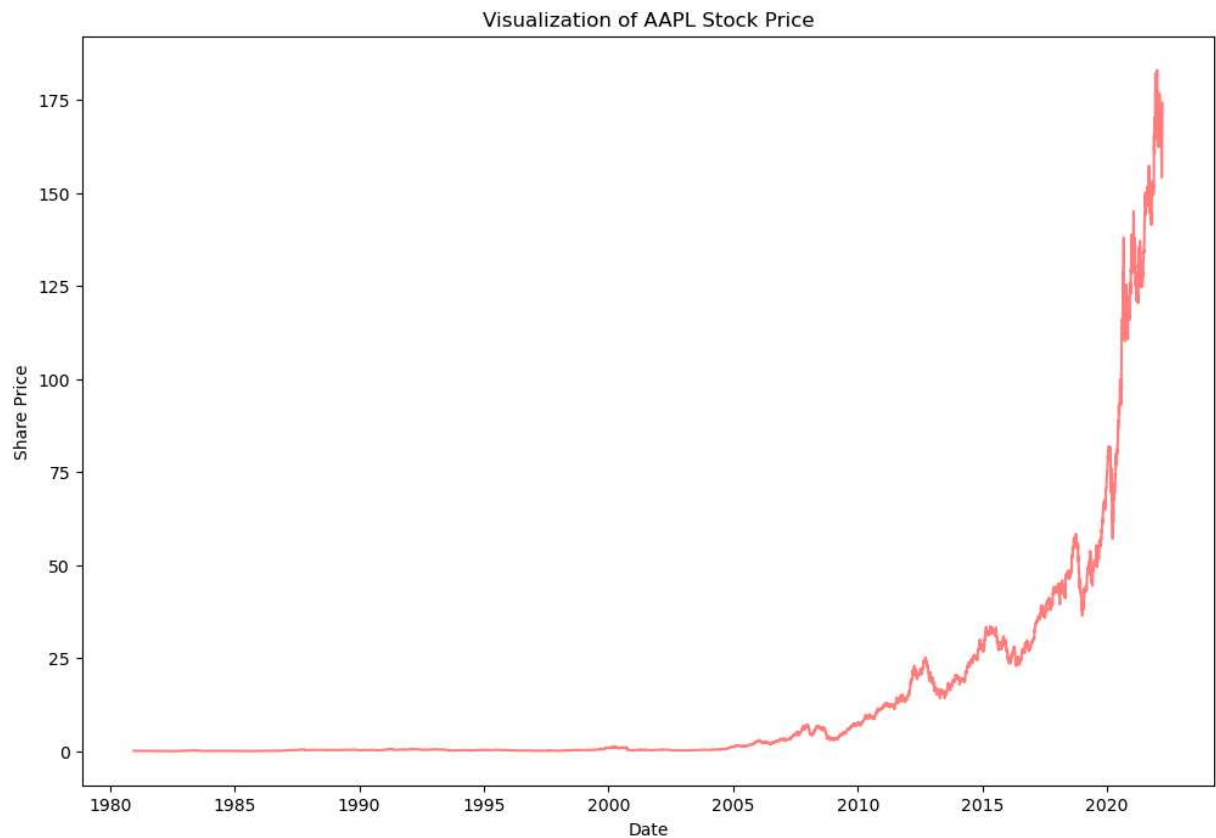
```

In [7]: #Import apple stock data
aapl = pd.read_csv('../data/AAPL.csv')

#Drop rows with missing values and convert date to datetime format
aapl['Date'] = pd.to_datetime(aapl['Date'])
aapl = aapl.dropna()

#Visualize the stock price
basic_stock_plot(aapl, 'AAPL')

```



```
In [8]: #Select features and target values
aapl_x = aapl[['Open', 'Low', 'Close', 'Adj Close', 'Volume']].head(-1)
aapl_x = aapl_x.values
aapl_y = aapl[['High']].tail(-1)
aapl_y = aapl_y.values

#Make predictions on aapl high price using previously trained Linear model
aapl_predicted_high_prices = model.predict(aapl_x)

#Print Comparison Table For Apple Price
comp_table(aapl_y, aapl_predicted_high_prices, 'AAPL')
```

AAPL Prediction Comparison

	Actual	Predicted
0	0.122210	126.202348
1	0.113281	46.617936
2	0.116071	27.565117
3	0.119420	22.332120
4	0.126674	18.808301
...	...	...
10403	164.479996	181.203376
10404	166.350006	197.572719
10405	169.419998	191.475676
10406	172.639999	191.093672
10407	174.139999	196.959297

[10408 rows x 2 columns]

```
In [9]: # Convert aapl_predicted_high_prices to a 1D array if it's 2D
aapl_predicted_high_prices = aapl_predicted_high_prices.flatten()
```

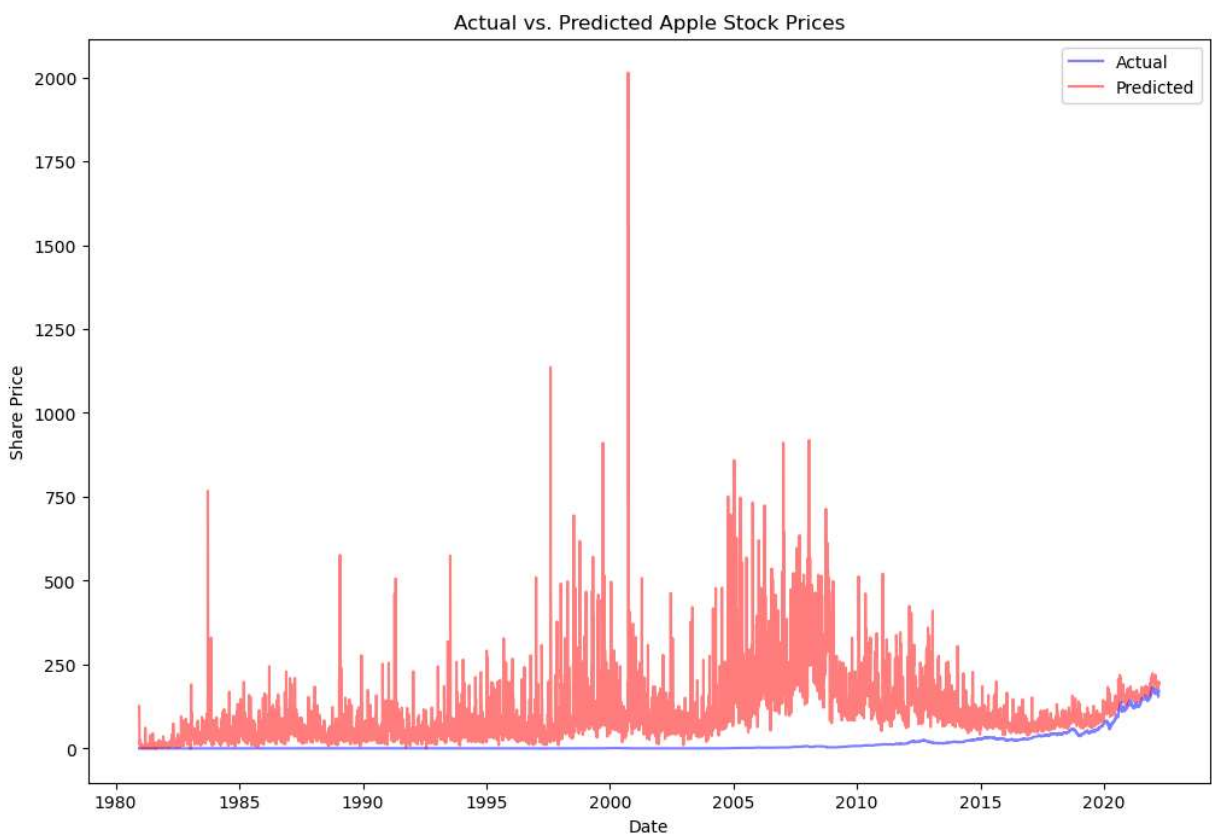
```

# Ensure both arrays have the same length (trim the larger array if needed)
min_length = min(len(aapl['Date']), len(aapl_predicted_high_prices))
dates = aapl['Date'][:min_length]
actual_prices = aapl['High'][:min_length]
predicted_prices = aapl_predicted_high_prices[:min_length]

# Plot the data
plt.figure(figsize=(12, 8))
plt.plot(dates, actual_prices, color='blue', alpha=0.5)
plt.plot(dates, predicted_prices, color='red', alpha=0.5)

plt.title('Actual vs. Predicted Apple Stock Prices')
plt.xlabel('Date')
plt.ylabel('Share Price')
plt.legend(['Actual', 'Predicted'])
plt.show()

```



```

In [10]: # Calculate the mean squared error - very high value indicating poor model fit
mse = mean_squared_error(actual_prices, predicted_prices)
print('AAPL Mean Squared Error:', mse)

# Calculate the r2 score - negative score indicating poor model fit
print('AAPL testing R2 Score:', r2_score(actual_prices, predicted_prices))

```

AAPL Mean Squared Error: 16328.500435042348
 AAPL testing R2 Score: -16.582160847135285

```

In [11]: # Import voo (Vanguard S&P 500 ETF) stock data
voo = pd.read_csv('../data/V00.csv')

```

```

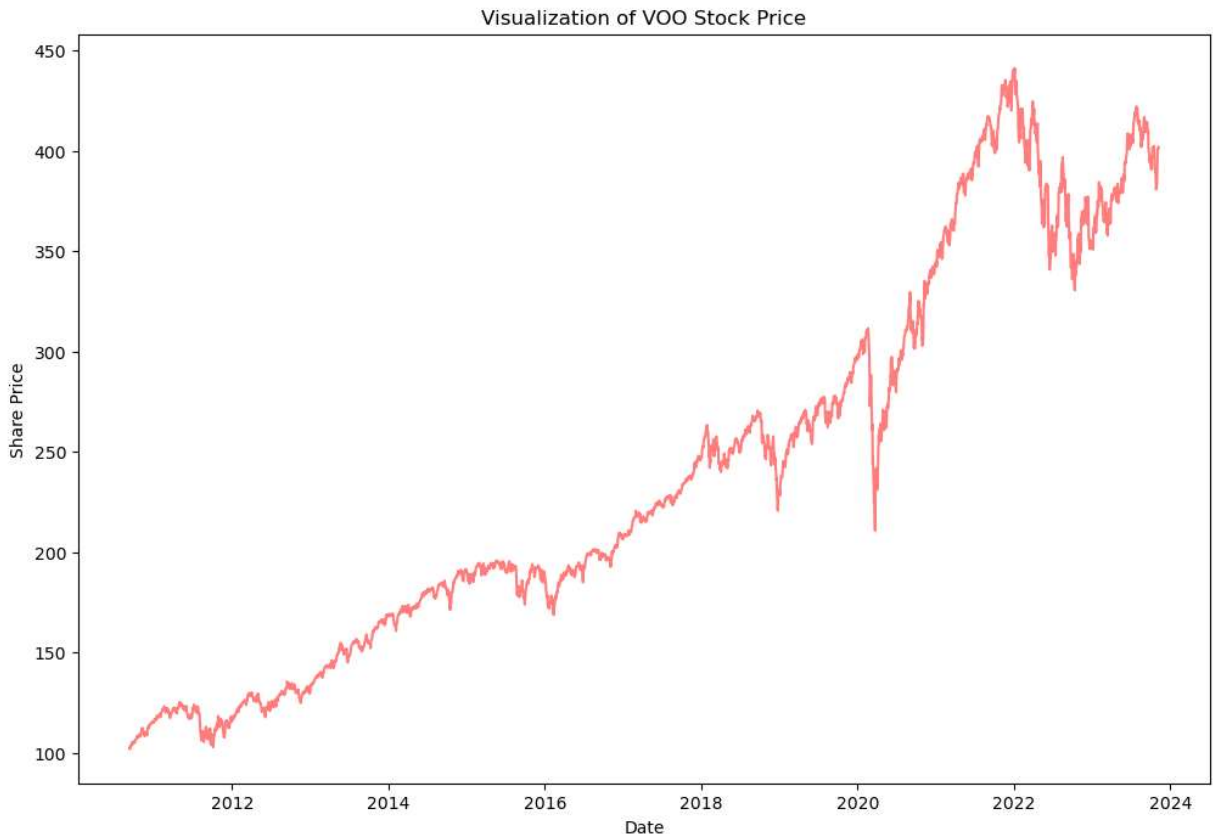
#Reorder indices to be consistent with past datasets
voo[:,] = voo[::-1]

#Drop rows with missing values, convert date to datetime format, strip whitespace f
voo = voo.dropna()
voo['Date'] = pd.to_datetime(voo['Date'], format = '%m/%d/%y')
voo.columns = voo.columns.str.strip()

#Insert new column 'Adj Close' so number of features matches what model expects. (0
voo.insert(5, 'Adj Close', voo['Close'], True)

basic_stock_plot(voo, 'VOO')

```



```

In [12]: #Select features and target values
voo_x = voo[['Open', 'Low', 'Close', 'Adj Close', 'Volume']].head(-1)
voo_x = voo_x.values
voo_y = voo[['High']].tail(-1)
voo_y = voo_y.values

#Make predictions on voo high price with previously trained linear model
voo_predicted_high_prices = model.predict(voo_x)

```

```

In [13]: # Convert voo_predicted_high_prices to a 1D array if it's 2D
voo_predicted_high_prices = voo_predicted_high_prices.flatten()

# Ensure both arrays have the same length (trim the Larger array if needed)
min_length = min(len(voo['Date']), len(voo_predicted_high_prices))
dates = voo['Date'][:min_length]
actual_prices = voo['High'][:min_length]
predicted_prices = voo_predicted_high_prices[:min_length]

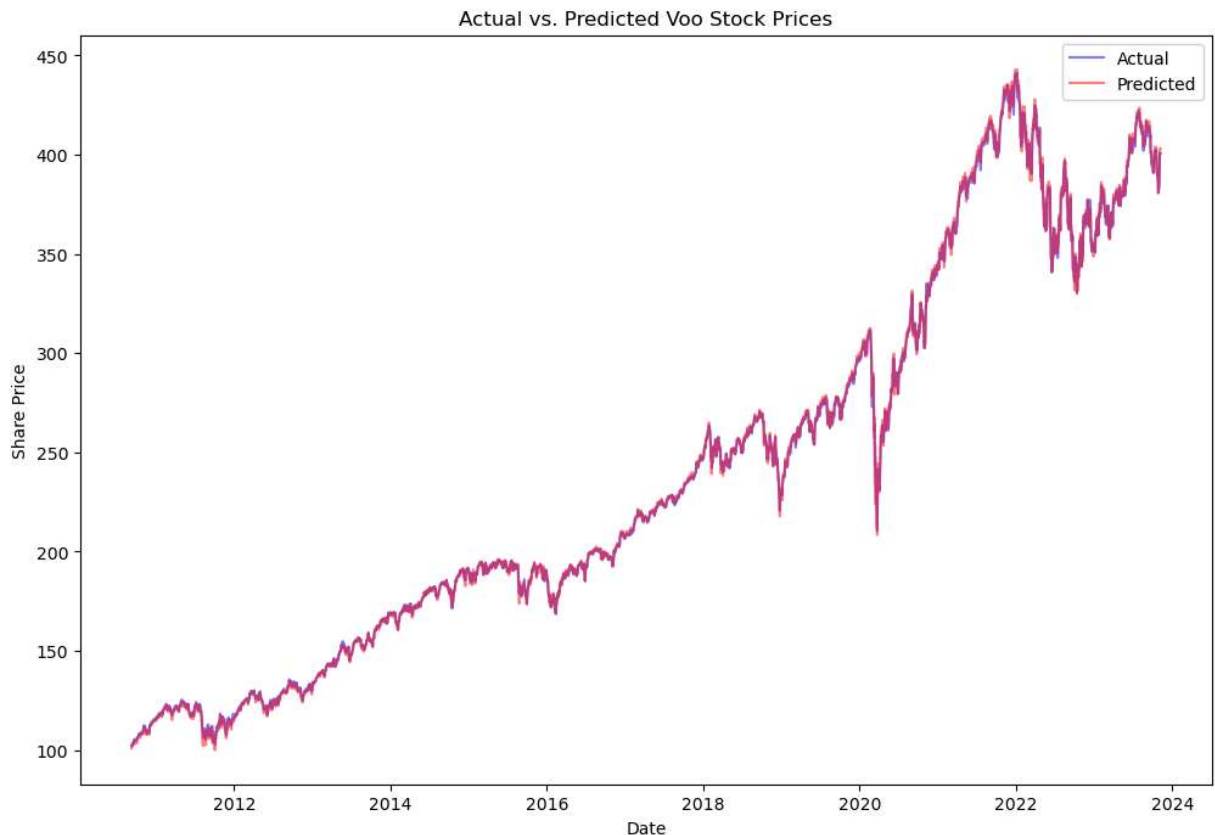
```

```

# Plot the data
plt.figure(figsize=(12, 8))
plt.plot(dates, actual_prices, color='blue', alpha=0.5)
plt.plot(dates, predicted_prices, color='red', alpha=0.5)

plt.title('Actual vs. Predicted Voo Stock Prices')
plt.xlabel('Date')
plt.ylabel('Share Price')
plt.legend(['Actual', 'Predicted'])
plt.show()

```



```

In [14]: # Calculate the mean squared error
mse = mean_squared_error(actual_prices, predicted_prices).round(4)
print('V00 Prediction MSE:', mse)

#Calculate the r2 score
r2_voo = round(r2_score(actual_prices, predicted_prices), 4)
print(f'V00 Prediction R2:', r2_voo)

#Print Comparison Table For V00 Prices
comp_table(voo_y, voo_predicted_high_prices, 'V00')

```

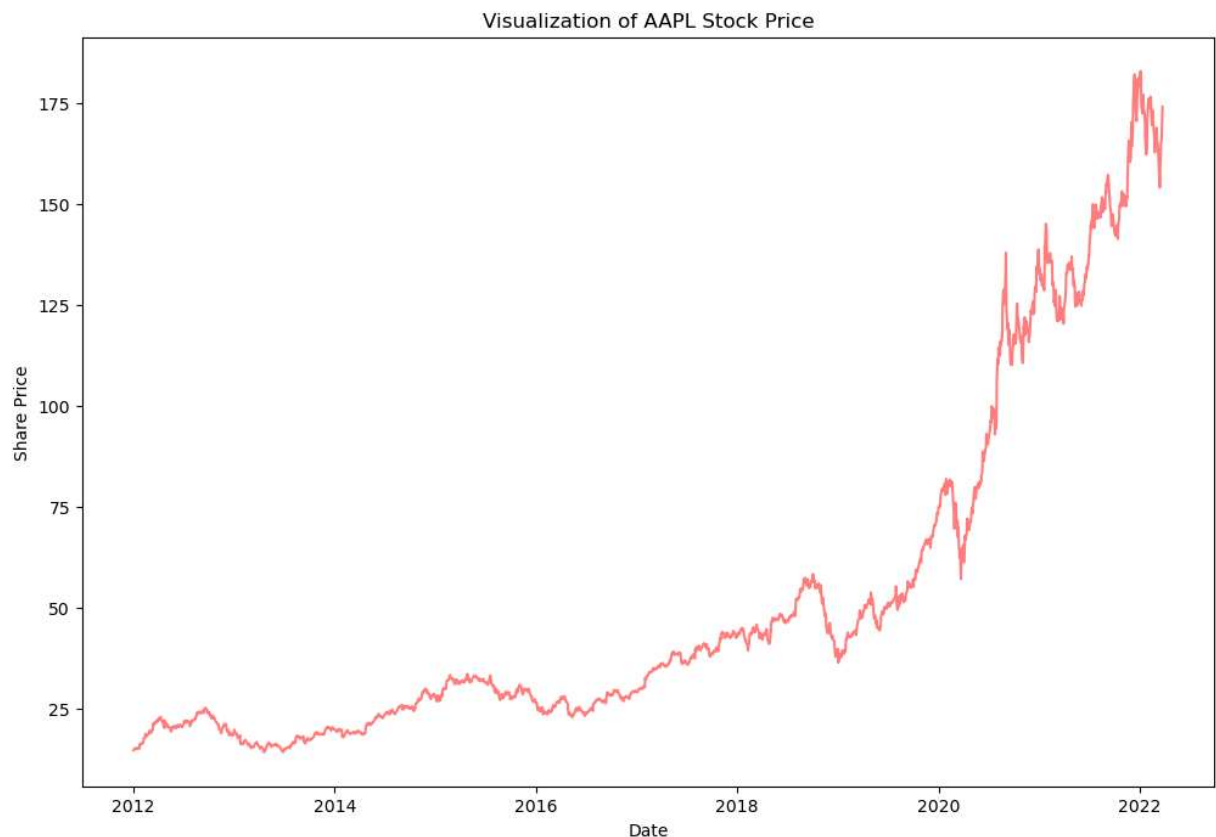
V00 Prediction MSE: 2.687
V00 Prediction R2: 0.9997
V00 Prediction Comparison

	Actual	Predicted
0	101.8600	100.868038
1	103.1400	101.341279
2	103.4800	102.638750
3	103.3800	102.625360
4	103.3200	102.882449
...	...	...
3308	389.0800	387.194606
3309	395.9096	391.751149
3310	400.8500	398.977776
3311	400.7330	402.957446
3312	402.0400	403.185844

[3313 rows x 2 columns]

```
In [15]: #Trim AAPL dataset to see if reducing number of outliers improves predictions
trimmed_aapl_df = aapl[(aapl['Date'] >= (pd.to_datetime('2012-01-01')))]

#Visualize basic plot of trimmed AAPL data
basic_stock_plot(trimmed_aapl_df, 'AAPL')
```



```
In [16]: #Select features and target values
aapl_x_trimmed = trimmed_aapl_df[['Open', 'Low', 'Close', 'Adj Close', 'Volume']].h
aapl_x_trimmed = aapl_x_trimmed.values
aapl_y_trimmed = trimmed_aapl_df[['High']].tail(-1)
aapl_y_trimmed = aapl_y_trimmed.values
```

```
#Make predictions on aapl trimmed high price with previously trained Linear model
aapl_trimmed_predicted_high_prices = model.predict(aapl_x_trimmed)

#Print Comparison Table For AAPL trimmed Prices
comp_table(aapl_y_trimmed, aapl_trimmed_predicted_high_prices, 'AAPL Trimmed')
```

AAPL Trimmed Prediction Comparison

	Actual	Predicted
0	14.810000	94.523265
1	14.948214	83.142837
2	15.098214	86.349518
3	15.276786	99.259369
4	15.214286	119.792158
...	...	...
2568	164.479996	181.203376
2569	166.350006	197.572719
2570	169.419998	191.475676
2571	172.639999	191.093672
2572	174.139999	196.959297

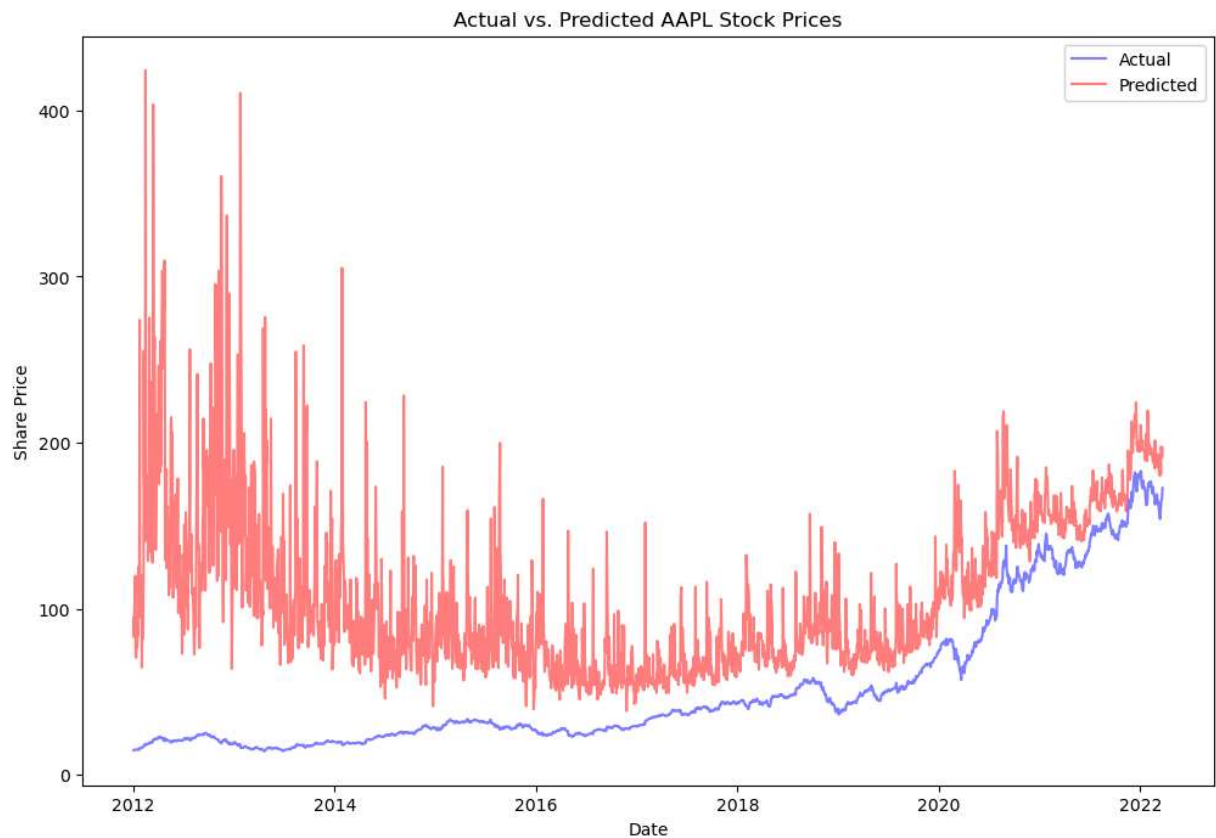
[2573 rows x 2 columns]

```
In [17]: # Convert aapl_trimmed_predicted_high_prices to a 1D array if it's 2D
aapl_trimmed_predicted_high_prices = aapl_trimmed_predicted_high_prices.flatten()

# Ensure both arrays have the same length (trim the larger array if needed)
min_length = min(len(trimmed_aapl_df['Date']), len(aapl_trimmed_predicted_high_prices))
dates = trimmed_aapl_df['Date'][:min_length]
actual_prices = trimmed_aapl_df['High'][:min_length]
predicted_prices = aapl_trimmed_predicted_high_prices[:min_length]

# Plot the data
plt.figure(figsize=(12, 8))
plt.plot(dates, actual_prices, color='blue', alpha=0.5)
plt.plot(dates, predicted_prices, color='red', alpha=0.5)

plt.title('Actual vs. Predicted AAPL Stock Prices')
plt.xlabel('Date')
plt.ylabel('Share Price')
plt.legend(['Actual', 'Predicted'])
plt.show()
```



```
In [18]: # Calculate the mean squared error
mse = mean_squared_error(actual_prices, predicted_prices)
print('AAPL Mean Squared Error:', mse)

#Calculate the r2 score
print('AAPL testing R2 Score:', r2_score(actual_prices, predicted_prices))
```

AAPL Mean Squared Error: 5299.239685300573

AAPL testing R2 Score: -1.9329636417941303

```
In [19]: #Training a new linear model based on the original AAPL data

#Select the following columns to be features and select all rows except very last row
x = aapl[['Open', 'Low', 'Close', 'Adj Close', 'Volume']].head(-1)
x = x.values

#Select 'High' price to be target value. Select all but first row.
y = aapl[['High']].tail(-1)
y = y.values

#Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

#Initialize AAPL linear model and fit training data to model
aapl_model = LinearRegression()
aapl_model.fit(x_train, y_train)

#Make predictions on AAPL testing data
predicted_high_prices = aapl_model.predict(x_test)
```

```

#Printing R2 score for training data.
print('AAPL training data R2 Score:', aapl_model.score(x_train, y_train))

#Print R2 Score comparing predicted vs actual prices.
print('AAPL testing data R2 Score:', r2_score(y_test, predicted_high_prices))

#Print new AAPL Comparison Table
print()
comp_table(y_test, predicted_high_prices, 'AAPL')

```

AAPL training data R2 Score: 0.9997769035110509

AAPL testing data R2 Score: 0.9997013724328174

AAPL Prediction Comparison

	Actual	Predicted
0	0.196429	0.194227
1	0.353237	0.352984
2	42.750000	42.832053
3	32.375000	32.381941
4	0.140625	0.145277
...	...	...
2077	0.325893	0.324153
2078	1.053571	1.062051
2079	32.657501	32.699296
2080	0.127790	0.128918
2081	0.483817	0.498411

[2082 rows x 2 columns]